

OFA-MAS: One-for-All Multi-Agent System Topology Design based on Mixture-of-Experts Graph Generative Models

Shiyuan Li*
Griffith University
Gold Coast, Australia
shiyuan.li@griffithuni.edu.au

Yixin Liu*
Griffith University
Brisbane, Australia
yixin.liu@griffith.edu.au

Yu Zheng
Griffith University
Gold Coast, Australia
yu.zheng@griffith.edu.au

Mei Li
Northwest A&F University
Shaanxi, China
limei@nwsuaf.edu.cn

Quoc Viet Hung Nguyen
Griffith University
Gold Coast, Australia
henry.nguyen@griffith.edu.au

Shirui Pan[†]
Griffith University
Gold Coast, Australia
s.pan@griffith.edu.au

Abstract

Multi-Agent Systems (MAS) offer a powerful paradigm for solving complex problems, yet their performance is critically dependent on the design of their underlying collaboration topology. As MAS become increasingly deployed in web services (e.g., search engines), designing adaptive topologies for diverse cross-domain user queries becomes essential. Current graph learning-based design methodologies often adhere to a “one-for-one” paradigm, where a specialized model is trained for each specific task domain. This approach suffers from poor generalization to unseen domains and fails to leverage shared structural knowledge across different tasks. To address this, we propose OFA-MAS, a one-for-all framework that generates adaptive collaboration graphs for any task described in natural language through a single universal model. Our approach integrates a Task-Aware Graph State Encoder (TAGSE) that filters task-relevant node information via sparse gating, and a Mixture-of-Experts (MoE) architecture that dynamically selects specialized sub-networks to drive node and edge prediction. We employ a three-stage training strategy: unconditional pre-training on canonical topologies for structural priors, large-scale conditional pre-training on LLM-generated datasets for task-topology mappings, and supervised fine-tuning on empirically validated graphs. Experiments across six diverse benchmarks show that OFA-MAS significantly outperforms specialized one-for-one models, generating highly adaptive MAS topologies. Code: <https://github.com/Shiy-Li/OFA-MAS>.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems; Neural networks**; • **Mathematics of computing** → **Graph algorithms**.

Keywords

Multi-Agent Systems, Large Language Models, Graph Neural Networks, Graph Generation

*Both authors contributed equally to the paper

[†]Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792537>

ACM Reference Format:

Shiyuan Li, Yixin Liu, Yu Zheng, Mei Li, Quoc Viet Hung Nguyen, and Shirui Pan. 2026. OFA-MAS: One-for-All Multi-Agent System Topology Design based on Mixture-of-Experts Graph Generative Models. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792537>

1 Introduction

Multi-agent systems (MAS) built upon large language models have emerged as a powerful approach for solving complex problems by orchestrating multiple specialized agents with distinct roles [9, 21, 23, 25]. To date, MAS have been increasingly integrated with web services, such as search engines, online knowledge bases, and cloud-based task execution platforms [12, 31, 38, 48]. However, the effectiveness of MAS highly depends on their *collaboration topology*, i.e., the structural design determining which agents communicate, when they interact, and how knowledge propagates [28, 36, 43]. An optimal topology applies the right expertise at the right moment, while poor topology design can lead to coordination inefficiencies that undermine performance.

To address this challenge of topology design, early efforts focus on manual designs like sequential chains [35, 44], tree hierarchies [37], and debate structures have shown domain-specific success [7, 19]. However, such manually crafted structures are often rigid and hard to generalize, since a topology that works well in one domain may fail to adapt to new tasks [14, 18]. More recently, *automatic topology design* emerged as a promising solution, which aims to adaptively construct agent interaction structures for a given query using deep graph learning models. For example, a line of methods constructs topologies by pruning predefined dense graphs [34, 42]. Another line of methods leverages GNN message passing [13, 17, 26] to adaptively learn task-specific interaction topologies for collaboration graphs [3, 30, 43, 46, 48].

Despite the effectiveness of the existing graph-based automatic topology design methods, these methods mainly focus on a “**one-for-one**” learning paradigm, that is, a topology design model is trained on a specific task dataset (e.g., MMLU) and can only generalize within these tasks. Such a paradigm, unfortunately, introduces critical barriers to the practical and large-scale deployment of these methods, which can be characterized by three shortcomings: (1) **Impractical Domain Assumptions**: The one-for-one

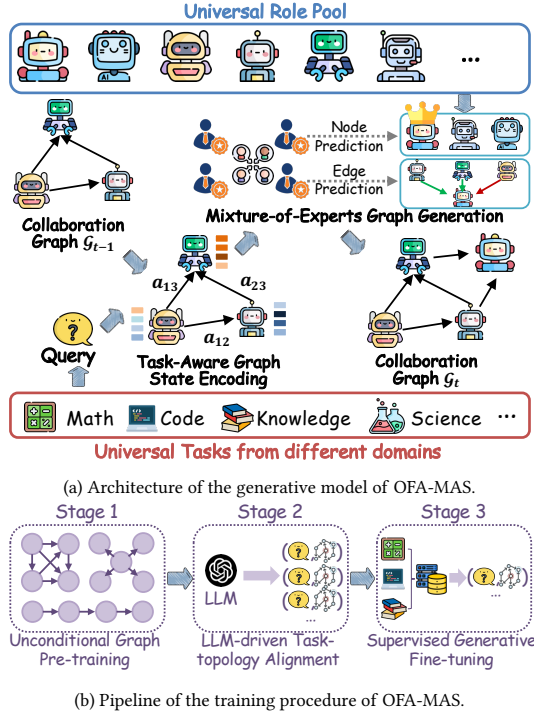


Figure 1: Sketch maps of OFA-MAS.

paradigm presumes optimization for a single and pre-defined task domain, yet real-world agentic systems in web services face user queries spanning unpredictable domains with cross-domain reasoning. Since general-purpose MAS cannot reasonably expect users to pre-categorize their requests, the domain constraint limits the application of the current methods to real-world user-facing services. (2) **Intractable Scalability and Maintenance:** While training multiple specialist models for each new domain may alleviate the domain limitation, its prohibitive overhead that scales with the increasing number of supported tasks compounds the difficulty. Each domain requires a dedicated pipeline for data collection, training, and tuning, rendering the approach unsustainable for broad applications, which creates an intractable maintenance burden in practical deployment. (3) **Overlooking Cross-Domain Shared Knowledge:** Learning one-for-one design models overlooks the opportunity to discover and leverage shared design patterns across domains. For instance, domains like mathematical theorem proving and software debugging often share the same underlying collaboration patterns (e.g., a “Analyst → Inspector → Solver” workflow). Training isolated models may hinder us from learning such abstract and reusable problem-solving knowledge, leading to redundant training efforts and suboptimal performance.

To overcome these limitations, a promising solution is to develop a “one-for-all” approach, designing a universal topology designer capable of generating optimal collaboration topologies across any task domain. A one-for-all design approach can naturally break the domain-specific limitation due to its generalization capability. Moreover, it can also increase scalability in practical scenarios by only maintaining a single universal topology designer instead of

training multiple domain-specific models. More importantly, the underlying shared knowledge across domains can be exploited to improve problem-solving efficiency.

In spite of its promising potential, developing a one-for-all method is a non-trivial task, facing two critical challenges. **Challenge 1 - Model Architecture Design:** Current topology design models typically rely on predefined and domain-specific role pools and lack mechanisms to incorporate task semantics into role selection. As a result, they often yield sub-optimal agent configurations when task demands differ across domains. Furthermore, traditional one-shot generation mechanisms hinder effective specialization, potentially leading to degraded performance when adapted to diverse task types. In this case, the architecture of one-for-all models should incorporate task-aware encoding mechanisms to capture global task context during topological design, while deploying an expert specialization framework that dynamically activates domain-specific expertise and preserves cross-task knowledge sharing. **Challenge 2 - Training Strategy:** To train a universal model for topology design, a prerequisite is large-scale and high-quality training data from multiple domains. However, collecting sufficient task queries with ground-truth optimal topologies is prohibitively expensive and labor-intensive. Moreover, even if such data were available, naively training on mixed-domain data could lead to catastrophic forgetting, where learning new domains degrades performance on previously learned ones. Therefore, an advanced training strategy should address the limitation of scarce data and design a learning curriculum that promotes effective cross-domain generalization.

Aiming to address the above challenges, we propose OFA-MAS, a unified graph generative framework that combines task-adaptive model architecture with a multi-stage training strategy for universal MAS topology design. To overcome **Challenge 1**, we develop an autoregressive graph generation model for MAS design with two designs (See Figure 1a). To extract task-related information to guide topology design, we propose a *task-aware graph state encoder* (TAGSE) that employs context-gated message passing to filter and regulate node representations by task relevance. Meanwhile, we design a *mixture-of-experts (MoE) graph generation module* to capture diverse collaboration patterns across domains with multiple experts, routed by an expert gating network to contribute to topology generation. To address **Challenge 2**, we design an *easy-to-hard three-stage training strategy* consisting of unconditional pre-training, LLM-guided conditional training, and supervised fine-tuning (See Figure 1b). The well-designed learning curriculum gradually introduces task consistency and preserves structural generation knowledge, mitigating catastrophic forgetting and enhancing cross-domain generalization. To mitigate data limitation, we propose a LLM-driven data synthesis pipeline that constructs multi-domain “query-topology” pair data with reduced annotation cost. Overall, our work makes the following contributions:

- **Practical Paradigm:** We pioneer the transition from the “one-for-one” to “one-for-all” paradigm in graph-based MAS topology design, establishing the first universal framework for cross-domain collaboration graph generation.
- **Novel Method:** We propose OFA-MAS, an autoregressive graph generation framework with TAGSE and a MoE module, trained via a three-stage curriculum with LLM-driven data synthesis.

- **Sufficient Experiments:** Experiments on six multi-domain benchmarks demonstrate that OFA-MAS outperforms existing specialized methods with strong generalization, significantly advancing universal autonomous agent systems.

2 Problem Formulation

2.1 Formulate MAS as Graphs

A MAS constitutes a collaborative network of autonomous agents that collectively solve complex problems through structured interactions and coordinated reasoning. In the context of LLMs, we formalize a MAS as a structured collaboration network $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where the topology explicitly defines both the participating agents and their interaction patterns. Each vertex $v_i \in \mathcal{V}$ corresponds to an agent instantiated from an LLM, characterized by two key attributes: a *functional role* ρ_i that determines its specialized expertise (e.g., Analyst, Programming Expert, Inspector), and a *contextual state* σ_i that encapsulates its accumulated knowledge from previous interactions. The edge set $\mathcal{E} \subset \mathcal{V} \times \mathcal{V}$ encodes the information flow topology as a directed acyclic graph (DAG), where each edge $(v_j, v_i) \in \mathcal{E}$ establishes a communication channel from agent v_j to agent v_i . We denote the set of immediate information providers for agent v_i as $\Psi(v_i) = \{v_j \mid (v_j, v_i) \in \mathcal{E}\}$.

The execution protocol follows a topologically-ordered activation sequence over multiple communication rounds, ensuring information dependencies are respected. The system can execute for K rounds to enable iterative refinement. In round k , agent v_i generates its message $\omega_i^{(k)}$ by invoking its underlying language model:

$$\omega_i^{(k)} = \text{LLM}_i(\text{Prompt}(\rho_i, \sigma_i, \{\omega_j^{(k-1)} \mid v_j \in \Psi(v_i)\})). \quad (1)$$

The system’s final output aggregates the final-round messages according to a task-appropriate combination strategy.

2.2 From One-for-One to One-for-All Paradigm

One-for-One MAS Design. Traditional graph-based MAS topology design [30, 43] follows a domain-specific paradigm where individual design models are developed for single task domains. Formally, given a task domain $\mathcal{D}$ characterized by a distribution of queries $Q \sim \mathcal{D}$ and a predefined role set $\mathcal{R}_{\mathcal{D}}$, the goal is to learn a domain-specific mapping function $f_{\mathcal{D}} : (Q, \mathcal{R}_{\mathcal{D}}) \rightarrow \mathcal{G}$ that generates optimal collaboration topologies for queries within that domain. In the conventional setting of “one model for one domain”, the design model $f_{\mathcal{D}}$ is optimized exclusively on the target domain dataset $\mathcal{T}_{\mathcal{D}} = \{(Q_i, \mathcal{G}_i^*)\}_{i=1}^{|\mathcal{T}_{\mathcal{D}}|}$, where each pair $(Q_i, \mathcal{G}_i^*)$ represents a query and its corresponding optimal topology. After sufficient training on $\mathcal{T}_{\mathcal{D}}$, the specialized model $f_{\mathcal{D}}$ can generate effective topologies for new queries from the same domain $\mathcal{D}$ during inference. While effective within individual domains, this approach has inherent limitations in cross-domain transferability and scalability. **One-for-All (OFA) MAS Design.** To address these limitations, we propose the **one-for-all graph-based MAS design**, where a single model capable of generating optimal collaboration topologies for diverse task domains without requiring domain-specific re-training. Formally, let $\mathcal{D}_{\text{train}} = \{\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_N\}$ be a collection of training domains, where each domain $\mathcal{D}_i$ (with $i \in \{1, \dots, N\}$) is associated with training and testing splits: $\mathcal{T}_i^{\text{train}} = \{(Q_j^{(i)}, \mathcal{G}_j^{*(i)})\}_{j=1}^{|\mathcal{T}_i^{\text{train}}|}$

and $\mathcal{T}_i^{\text{test}}$, where j indexes the query-topology pairs within each domain. We aim to train an OFA designer f_{OFA} on the combined multi-domain training data $\mathcal{T}_{\text{train}} = \bigcup_{i=1}^N \mathcal{T}_i^{\text{train}}$, so that f_{OFA} can generate effective topologies for queries across all test splits $\mathcal{T}_{\text{test}} = \bigcup_{i=1}^N \mathcal{T}_i^{\text{test}}$, where $\mathcal{T}_{\text{train}} \cap \mathcal{T}_{\text{test}} = \emptyset$.

Unlike traditional one-for-one graph-based approaches that require separate models for each domain, our OFA formulation enables a single model to handle diverse task types simultaneously. During inference, the model can generate appropriate topologies for any query from the supported domains without requiring domain-specific fine-tuning or model selection.

3 Methodology

In this section, we introduce OFA-MAS, a one-for-all MAS topology design approach, with the pipeline demonstrated in Figure 2. We employ an **autoregressive graph generation model** as the backbone of OFA-MAS (Section 3.1), which balances the performance and flexibility for the MAS design problem. Building upon the backbone, we design two modules to better support the one-for-all topology design: **task-aware graph state encoder** (Section 3.2) that maintains task-dependent representations of the partially constructed graph for guiding topology generation, and **mixture-of-experts generation module** (Section 3.3) that adaptively routes experts for node and edge generation according to the contextual requirements of different task types. To optimize OFA-MAS, we carefully design a **three-stage training curriculum** with an **LLM-based data synthesis approach** (Section 3.4). All components are detailed in the following subsections.

3.1 Autoregressive Graph Generation Backbone

Under the “one-for-all” paradigm, a single model should serve diverse domains with varying collaboration patterns. Nevertheless, existing graph learning-based models typically fix scale and role allocation, with a limited capability in role adaptation and scale generalization [43]. Specifically, they cannot simultaneously accommodate different structural requirements across domains nor scale to handle task-specific variations within each domain.

To address these limitations, we reframe the topology design problem: instead of adapting fixed configurations, we propose generating customized graphs from scratch. This generative approach naturally aligns with the OFA paradigm, offering greater flexibility for constructing domain-specific structures and handling the scalability demands of diverse tasks. We achieve this by learning a **conditional graph generation model** $P(\mathcal{G} \mid Q, \mathcal{R})$ that directly captures the mapping from task queries to effective collaborative topologies. This model aims to find the optimal topology $\mathcal{G}^*$ that maximizes the learned distribution:

$$\mathcal{G}_{\text{gen}}^* = \arg \max_{\mathcal{G} \in \mathcal{G}} P(\mathcal{G} \mid Q, \mathcal{R}_{\text{OFA}}), \quad (2)$$

where Q is the task query, and $\mathcal{R}_{\text{OFA}}$ denotes the universal role pool spanning all domains.

To make this complex joint distribution $P(\mathcal{G} \mid Q, \mathcal{R}_{\text{OFA}})$ tractable, we employ an autoregressive graph generation framework that decomposes topology construction into a sequence of local decisions. Specifically, the generation process unfolds step-by-step: at each

step t , we first select an agent role v_t , then predict incoming edges $\mathcal{E}_t$ connecting existing agents to this new node. The complete generation probability is factorized as:

$$P(\mathcal{G}|Q) = \prod_{t=1}^{|\mathcal{V}|} [P(v_t|\mathcal{G}_{<t}, Q) \cdot P(\mathcal{E}_t|v_t, \mathcal{G}_{<t}, Q)], \quad (3)$$

where $\mathcal{G}_{<t}$ represents the partially constructed graph with the first $t-1$ nodes and their interconnections. The edge prediction can be further decomposed as:

$$P(\mathcal{E}_t|v_t, \mathcal{G}_{<t}, Q) = \prod_{j=1}^{t-1} P(e_{j \rightarrow t}|v_t, \mathcal{G}_{<t}, Q), \quad (4)$$

where $e_{j \rightarrow t}$ is a binary variable indicating whether an edge exists from node j to the newly added node t .

At each generation step t , our model makes two sequential decisions: **Role Selection**: Determine the most appropriate agent role v_t to add next, considering the current partial topology $\mathcal{G}_{<t}$ and the task requirements Q . This decision leverages learned patterns about which roles effectively complement existing agent configurations. **Connection Prediction**: For the newly added agent v_t , predict which existing agents should provide it with information by modeling the probability of each potential incoming edge. This captures learned dependencies about information flow patterns that facilitate effective collaboration.

The autoregressive formulation is particularly well-suited for the OFA paradigm, providing the following key advantages: (1) **Cross-domain Tractability**: Sequential decomposition of the graph generation process allows the model to organize different roles and connections in a flexible manner, which enables the model to generalize across domains with varying requirements; (2) **Variable-Size Graph Modeling**: The autoregressive generation is also flexible in terms of variable generated graph sizes and topologies, which can better fit the tasks with different levels of difficulty and heterogeneous topology demands.

While this autoregressive backbone provides a flexible foundation, realizing effective OFA topology design requires addressing two critical gaps: how to integrate task-specific semantic constraints into graph state encoding (Section 3.2), and how to enable specialized generation strategies for different task categories without sacrificing universality (Section 3.3). We address these challenges through two key architectural innovations.

3.2 Task-Aware Graph State Encoder

At each step of the autoregressive generation process, the model requires an in-depth and contextualized understanding of the partially constructed graph $\mathcal{G}_{<t}$ to make informed decisions about the next node or edge to add. Traditional graph neural networks (GNNs) typically ignore global task context during their message-passing updates, potentially missing crucial task-specific structural requirements [3, 24, 46]. To address this limitation, we design a specialized encoder that we term the **Task-Aware Graph State Encoder (TAGSE)**, which persistently integrates the task context into every layer of its message-passing computations.

Unified Semantic Representation. The foundation of task-aware graph encoding lies in obtaining informative representations of

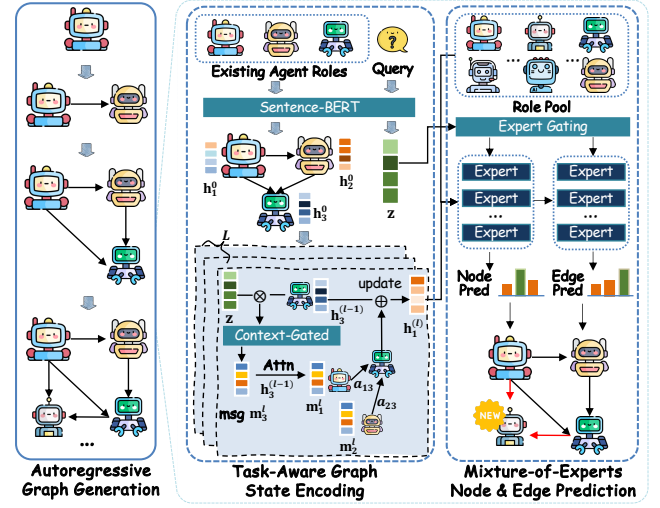


Figure 2: The workflow of OFA-MAS.

input task queries and role descriptions to guide the graph generation process. Since our OFA model needs to process queries across diverse domains, such as mathematical reasoning and code generation, we require a unified semantic space that enables cross-domain learning while retaining domain-specific features. To this end, we employ a dedicated encoding pipeline where a pre-trained sentence transformer first obtains a fixed-dimensional embedding e_Q for the given query Q , which is then projected into the final task vector $z \in \mathbb{R}^{d_{\text{task}}}$ through an MLP-based task encoder. This task vector z serves as the global conditioning signal that will be integrated into every layer of the subsequent graph encoding process. In addition, we establish node initial states $h_v^{(0)}$ by encoding agent role descriptions (e.g., Analyst and Programmer) using the same pre-trained sentence transformer, ensuring semantic consistency between task requirements and agent capabilities within the unified representation space. Given the initial states and task representation as input, for each layer $l \in \{1, \dots, L\}$, our encoder updates the node hidden states $h_v^{(l)}$ through the following mechanisms.

Context-Gated Message Passing. To ensure only task-relevant information flows between nodes, we employ a gating mechanism controlled by the task vector z that element-wise modulates the message from each node:

$$m_v^{(l)} = \sigma(W_g[h_v^{(l-1)} \| z]) \odot \text{ReLU}(W_m h_v^{(l-1)}), \quad (5)$$

where W_g and W_m are learnable weight matrices, $[\cdot \| \cdot]$ denotes concatenation, and $\odot$ represents element-wise multiplication. To encourage selective activation and prevent the gate from becoming overly dense, we apply L1 regularization to the gate values $\sigma(W_g[h_v^{(l-1)} \| z])$, promoting sparsity in the gating decisions and ensuring that only the most task-relevant information channels are activated.

Role-Aware Attention Aggregation. Given that different agent roles may require different types of information, we employ an attention mechanism that allows nodes to prioritize messages based on both the sender's current state and the receiver's processed

message:

$$e_{uv}^{(l)} = \text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}_k \mathbf{h}_u^{(l-1)} \parallel \mathbf{W}_q \mathbf{m}_v^{(l)}]), \quad (6)$$

$$\alpha_{uv}^{(l)} = \frac{\exp(e_{uv}^{(l)})}{\sum_{j \in \mathcal{N}_v} \exp(e_{jv}^{(l)})}, \quad (7)$$

where $\mathbf{W}_k, \mathbf{W}_q$, and $\mathbf{a}$ are learnable parameters, and $\mathcal{N}_v$ denotes the neighborhood of node v .

Residual State Update. To ensure stable training and preserve important information from previous layers, we employ a residual connection that combines the previous node state with the newly aggregated message information:

$$\mathbf{h}_v^{(l)} = \frac{1}{2} (\mathbf{h}_v^{(l-1)} + \hat{\mathbf{m}}_v^{(l)}), \quad (8)$$

where $\hat{\mathbf{m}}_v^{(l)} = \sum_{u \in \mathcal{N}_v} \alpha_{uv}^{(l)} \mathbf{m}_u^{(l)}$ represents the attention-weighted aggregated message.

The proposed TAGSE encoder deeply integrates task information into the message passing process, leading to sensitivity to queries from different tasks. Meanwhile, during aggregation, it considers both the correlations among connected roles and the ego information of each node state, which generates a comprehensive encoding for each agent. The high-quality embeddings learned by TAGSE provide an essential foundation for generation decisions.

3.3 Mixture-of-Experts Generation Module

A single and static GNN often struggles to fit diverse task types, as its parameters have to find a suboptimal compromise between different reasoning patterns. To enable strong versatility in topology generation, we employ a Mixture-of-Experts (MoE) paradigm that allows the model to dynamically reconfigure its computational pathways based on task characteristics. This module consists of two components, namely *expert gating network* and *MoE prediction head*, that work together to provide specialized and task-adaptive generation capabilities.

3.3.1 Expert Gating Network. The gating network serves as the decision-making component that determines which specialized experts should be activated for a given task. Taking the task vector $\mathbf{z}$ as input, a lightweight gating network computes a probability distribution over K available expert networks:

$$\mathbf{w} = \text{Softmax}(\text{MLP}_{\text{gate}}(\mathbf{z})), \quad (9)$$

where $\mathbf{w} \in \mathbb{R}^K$ represents the expert activation weights. This mechanism enables the model to learn distinct “design strategies” for different categories of tasks. For instance, mathematical reasoning tasks may activate experts that favor sequential chain topologies (e.g., “Analyst $\rightarrow$ Solver $\rightarrow$ Verifier”), while code debugging tasks may prefer hierarchical review structures (e.g., multiple Reviewers feeding into a central Debugger), and the model dynamically combines their insights based on the current task’s requirements.

3.3.2 MoE Prediction Heads. The following step in the autoregressive generation process leverages the task-aware graph representations and the expert selection weights to make precise predictions of graph structure expansion. The prediction heads operate in two stages corresponding to the two fundamental decisions required at each generation step:

Node Role Prediction. To determine the next agent role to add to the graph, we employ an MoE layer that processes the global graph representation. Each of the K expert networks produces its own prediction for the role distribution, and the final output combines these predictions using the gating weights:

$$P(\text{role}_t | \mathcal{G}_{<t}, \mathbf{z}) = \sum_{k=1}^K w_k \cdot \text{Expert}_k^{\text{node}}(\text{GlobalPool}(\mathcal{G}_{<t}), \mathbf{z}), \quad (10)$$

where $\text{GlobalPool}(\mathcal{G}_{<t})$ represents a global representation of the partial graph obtained through pooling operations over node representations.

Edge Connection Prediction. For each potential incoming edge to the newly added node, another set of expert networks predicts the connection probability. Taking the concatenation of the source node’s representation, the target node’s representation, and the task vector, each expert generates a prediction for the connectivity, and then the predictions are fused together to form the final probability:

$$P(\text{edge}_{j \rightarrow t} | \mathcal{G}_{<t}, \mathbf{z}) = \sum_{k=1}^K w_k \cdot \text{Expert}_k^{\text{edge}}([\mathbf{h}_j \parallel \mathbf{h}_t \parallel \mathbf{z}]), \quad (11)$$

where $\mathbf{h}_j$ and $\mathbf{h}_t$ represent the node representations of the source and target nodes, respectively.

This MoE-based architecture ensures that every generation decision results from a task-specific ensemble of specialized experts, enabling the model to adapt its generation strategy dynamically while maintaining consistency through the shared task representation.

3.4 Training Strategy and Data Construction

In order to train a versatile model like OFA-MAS, a large-scale set of high-quality ‘(task, optimal_graph)’ supervised data is highly required. Nevertheless, it is costly to collect such annotations due to the intensive human expertise required for structure design and the computational overhead of verifying structural validity. At the same time, directly training on heterogeneous multi-domain data may lead to degraded performance, as the model can suffer from catastrophic forgetting and unstable convergence. To overcome these limitations, we devise a three-stage training curriculum that bootstraps the model’s knowledge, evolving from general structural understanding to specific and high-quality task alignment. During the training process, we introduce an LLM-driven data synthesis module to alleviate the data limitation, enabling us to train OFA-MAS on large-scale “query-topology” data without human annotation. This strategy makes the training of an OFA generative model feasible by systematically building knowledge from low-cost and large-scale data before refining it with a small amount of empirically validated data.

3.4.1 Stage 1: Unconditional Graph Pre-training. A randomly initialized generative model has no intrinsic knowledge of what constitutes a valid or efficient graph structure. To address this, we first pre-train the model to learn the fundamental “grammar” of communication graphs for MAS, independent of any task. Specifically, we curate a dataset, $\mathcal{D}_{\text{graphs}}$, which includes classic communication topologies (e.g., Chain, Star, and FullConnected) with varying sizes

Table 1: Performance comparison (%). The best results are highlighted in bold, and the second-best results are underlined.

Method	MMLU	GSM8K	AQuA	MultiArith	SVAMP	HumanEval	Average
Direct Answer							
Vanilla	80.39	82.30	71.06	93.09	86.55	71.39	80.80
Single-Agent Prompting							
CoT	81.69 $\uparrow$ 1.30	86.50 $\uparrow$ 4.20	73.58 $\uparrow$ 2.52	93.25 $\uparrow$ 0.16	87.36 $\uparrow$ 0.81	74.67 $\uparrow$ 3.28	82.84 $\uparrow$ 2.04
SC (CoT)	83.66 $\uparrow$ 3.27	81.60 $\downarrow$ 0.70	75.63 $\uparrow$ 4.57	94.12 $\uparrow$ 1.03	88.59 $\uparrow$ 2.04	79.83 $\uparrow$ 8.44	83.91 $\uparrow$ 3.11
Fixed Multi-Agent Topologies							
Chain	83.01 $\uparrow$ 2.62	88.30 $\uparrow$ 6.00	74.05 $\uparrow$ 2.99	93.27 $\uparrow$ 0.18	87.17 $\uparrow$ 0.62	81.37 $\uparrow$ 9.98	84.53 $\uparrow$ 3.73
Tree	81.04 $\uparrow$ 0.65	85.20 $\uparrow$ 2.90	71.23 $\uparrow$ 0.17	93.68 $\uparrow$ 0.59	88.91 $\uparrow$ 2.36	80.53 $\uparrow$ 9.14	83.43 $\uparrow$ 2.63
Complete	82.35 $\uparrow$ 1.96	80.10 $\downarrow$ 2.20	72.95 $\uparrow$ 1.89	94.53 $\uparrow$ 1.44	84.01 $\downarrow$ 2.54	79.03 $\uparrow$ 7.64	82.16 $\uparrow$ 1.36
Random	84.31 $\uparrow$ 3.92	86.90 $\uparrow$ 4.60	76.48 $\uparrow$ 5.42	94.08 $\uparrow$ 0.99	87.54 $\uparrow$ 0.99	82.66 $\uparrow$ 11.27	85.33 $\uparrow$ 4.53
LLM-Debate	84.96 $\uparrow$ 4.57	91.40 $\uparrow$ 9.10	77.65 $\uparrow$ 6.59	96.36 $\uparrow$ 3.27	90.11 $\uparrow$ 3.56	84.70 $\uparrow$ 13.31	87.53 $\uparrow$ 6.73
Design Models with Dataset-Specific Training							
AgentPrune	85.07 $\uparrow$ 4.57	91.10 $\uparrow$ 8.80	80.51 $\uparrow$ 9.45	94.65 $\uparrow$ 1.56	90.58 $\uparrow$ 4.03	86.75 $\uparrow$ 15.36	88.09 $\uparrow$ 7.29
AgentDropout	85.62 $\uparrow$ 5.23	91.70 $\uparrow$ 9.40	80.94 $\uparrow$ 9.88	95.60 $\uparrow$ 2.51	91.04 $\uparrow$ 4.49	85.98 $\uparrow$ 14.59	88.48 $\uparrow$ 7.68
G-designer	86.92 $\uparrow$ 6.53	93.80 $\uparrow$ 11.50	81.60 $\uparrow$ 10.54	96.50 $\uparrow$ 3.41	93.10 $\uparrow$ 6.55	88.33 $\uparrow$ 16.94	90.04 $\uparrow$ 9.24
EIB-LEARNER	<u>88.90</u> $\uparrow$ 8.51	<u>95.20</u> $\uparrow$ 12.90	<u>83.49</u> $\uparrow$ 12.43	<u>96.83</u> $\uparrow$ 3.74	<u>94.70</u> $\uparrow$ 8.15	89.15 $\uparrow$ 17.76	91.38 $\uparrow$ 10.58
One-for-All Topology Design							
OFA-MAS (pre-trained)	<u>88.90</u> $\uparrow$ 8.51	94.90 $\uparrow$ 12.60	83.18 $\uparrow$ 12.12	99.11 $\uparrow$ 6.02	94.27 $\uparrow$ 7.72	<u>92.56</u> $\uparrow$ 21.17	<u>92.15</u> $\uparrow$ 11.35
OFA-MAS (fine-tuned)	89.54 $\uparrow$ 9.15	95.30 $\uparrow$ 13.00	85.05 $\uparrow$ 13.99	99.11 $\uparrow$ 6.02	94.90 $\uparrow$ 8.35	94.21 $\uparrow$ 22.82	93.02 $\uparrow$ 14.92

and role allocation, and then train OFA-MAS model autoregressively on this data. During this stage, the task vector $\mathbf{z}$ is replaced with a zero vector. This stage teaches the generator to learn how to construct valid and coherent graph structures based solely on the sequence of previous decisions, thereby establishing a strong structural prior before any task-specific knowledge is introduced.

3.4.2 Stage 2: Task-Topology Alignment via LLM-driven Data Synthesis. Training an OFA model requires extensive (query, topology) pairs spanning diverse domains to learn the mapping from task requirements to optimal graph structures. However, obtaining such data through actual MAS execution would be prohibitively expensive, as it requires running complete multi-agent systems for each query-topology pair to generate ground-truth labels. To address this challenge, we leverage a LLM as a “proxy system designer” to generate massive pseudo-labeled datasets at low cost. Concretely, we prompt the LLM to produce a large amount of diverse “(task query, MAS configuration)” pairs belonging to different domains; details are in Appendix D.2 The synthetic MAS configurations are then converted into a structured graph dataset $\mathcal{D}_{\text{gen}}$. To optimize OFA-MAS, we perform conditional pre-training on this synthetic dataset, starting from the model after Stage 1 training. This stage efficiently transfers the general-purpose reasoning ability of the LLM into our specialized model, teaching it the alignment patterns between task semantics and graph structures without requiring expensive MAS executions.

3.4.3 Stage 3: Supervised Generative Fine-tuning. While the knowledge acquired from the LLM-generated data is broad, it may not be optimal for the specific scenarios. To obtain high-quality training data, we leverage a small fraction of training data from real-world datasets with predefined classical topology configurations to execute MAS systems and obtain empirically validated performance

data. Specifically, for each query in the training sets across all domains, we apply established topological structures (e.g., sequential chains, layered, and star) and execute the corresponding MAS to evaluate their performance. We curate a dataset $\mathcal{D}_{\text{fine-tune}}$ consisting of “(task query, high-performing topology)” pairs where the topologies represent proven classical configurations that demonstrate superior performance through actual MAS execution. Finally, we perform fine-tuning on this empirically grounded dataset.

3.4.4 Training Objective. To optimize the autoregressive generation model, we define a loss function that captures the core goals of accurate generation, efficient use of the model’s expert capacity, and sparse gating behavior. The model is trained end-to-end in all stages by minimizing the loss:

$$L_{\text{total}} = L_{\text{graph}} + \lambda_{\text{balance}} L_{\text{balance}} + \lambda_{\text{gate}} L_{\text{gate}}, \quad (12)$$

where λ_{balance} and λ_{gate} are scalar hyperparameters that weight the contribution of each loss term. The primary supervised signal, L_{graph} , is calculated under a teacher forcing manner, and is used in all three training stages. It is composed of a node prediction loss and an edge prediction loss at each step of the generation:

$$L_{\text{graph}} = \sum_{t=1}^{|\mathcal{V}|} (L_{\text{node}}^{(t)} + L_{\text{edge}}^{(t)}), \quad (13)$$

where $L_{\text{node}}^{(t)}$ is the standard cross-entropy loss for the new node’s role, and $L_{\text{edge}}^{(t)}$ is the binary cross-entropy with logits loss for predicting the incoming edges. The auxiliary loss, L_{balance} , encourages the gating network to use all experts, ensuring the full capacity of the MoE architecture is utilized. The sparse gating regularization term, L_{gate} , applies L1 regularization to the gate activations in TAGSE, promoting selective information flow and preventing overly dense gating patterns that could lead to inefficient computation.

Table 2: Out-of-distribution generalization on GAIA benchmark (unseen dataset during training).

Method	Level-1	Level-2	Level-3	Average
Complete	5.66	8.14	0.00	4.60
Chain	11.32	6.98	3.85	7.38
AgentPrune	9.43	8.14	0.00	5.86
G-Designer	5.66	6.98	3.85	5.50
EIB-LEARNER	9.43	6.98	0.00	5.47
OFA-MAS	13.21	9.30	3.85	8.79

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate on MMLU [8], GSM8K [5], AQuA [16], MultiArith [29], SVAMP [27], and HumanEval [2], and test OOD generalization on GAIA [20]; details are in Appendix B.

Baselines. We compare OFA-MAS with individual-agent prompting (CoT [35], Self-Consistency [33]), static topologies, debate-based systems, and one-for-one graph learning-based designers (AgentPrune [42], AgentDropout [34], G-Designer [43], EIB-LEARNER [30]); details are in Appendix C.

Evaluation and Implementation. We report the success rate on each test set, and train a single unified OFA-MAS model across all datasets while training one-for-one baselines per domain. Implementation details are in Appendix D.

4.2 Main Results

The main performance comparison is presented in Table 1. We conduct the comparison with two versions of OFA-MAS: ❶ OFA-MAS (pre-trained), which is only trained on synthetic data (Stage 1 and Stage 2) without any real or benchmark-specific supervision; ❷ OFA-MAS (fine-tuned), which is trained following the full three-stage curriculum, including fine-tuning on real benchmark data.

According to the results, we have the following observations.

❶ **OFA-MAS achieves a state-of-the-art average score of 93.02%**, outperforming all baselines by a significant margin. This highlights the effectiveness of our one-for-all approach, which learns to generate specialized collaboration topologies for diverse tasks from a single, unified model. ❷ **Fixed topologies and generic prompting show sub-optimal performance.** We can see that fixed MAS topologies such as *Chain* (84.53%) provide a substantial performance boost, confirming that structured agent collaboration is crucial. However, no single fixed topology excels across all tasks. For instance, *Chain* performs well on sequential tasks like HumanEval, while *Complete* struggles. ❸ **OFA-MAS significantly outperforms existing learning-based MAS design methods.** For example, it surpasses the strong EIB-LEARNER baseline by 1.64% points. This result validates that a “one-for-all” model, trained across multiple domains, can leverage shared structural knowledge to outperform specialized “one-for-one” models. ❹ **OFA-MAS can achieve competitive performance even without fine-tuning on real-world data.** OFA-MAS (pre-trained) achieves a remarkable average performance of 92.15%, surpassing the best-performing baseline, EIB-LEARNER (91.38%). This demonstrates the effectiveness of our synthetic data-based pre-training strategy.

Table 3: Results of ablation study.

Method	MMLU	AQuA	HumanEval	Average
OFA-MAS	89.54	85.05	94.21	89.60
w/o MoE	87.58	84.11	90.91	87.53
w/o TASE	88.24	83.64	90.91	87.60
w/o Stage 1	88.24	84.58	90.08	87.08
w/o Stage 2	85.62	84.11	90.91	86.88
w/o L_{balance}	88.24	83.64	88.43	86.77
w/o L_{gate}	86.27	81.78	93.39	87.15

4.3 Out-of-Distribution Generalization

To evaluate generalization to unseen domains, we test all methods on GAIA, which was never seen during training. For fair evaluation, GPT-4o-mini is used as the base model without applying agentic techniques (e.g., tool calling), making the GAIA performance not directly comparable with commercial frameworks. Table 2 presents results across three difficulty levels. We observe: ❶ **OFA-MAS demonstrates strong zero-shot transfer capability**, achieving 8.79% average accuracy and outperforming all baselines. It excels on Level-1 tasks (13.21%), showing effective adaptation to new task types. ❷ **Learning-based baselines struggle with OOD tasks.** G-Designer only achieves 5.50% average accuracy, underperforming even the simple Chain topology (7.38%). This validates that our one-for-all paradigm learns more generalizable collaboration patterns than domain-specialized one-for-one approaches.

4.4 Ablation Study

To validate the effectiveness of our designs, we conduct a series of ablation studies, with results presented in Table 3. We have the following observations. ❶ **Our architectural innovations substantially improve topology generation.** Specifically, replacing these components with default modules leads to a significant performance drop, demonstrating the effectiveness of our tailored designs. ❷ **Both of the two pre-training stages (i.e., Stage 1 and 2) bring critical performance improvements.** We can see that removing each of them can bring significant degradation, demonstrating their effectiveness in OFA model optimization. ❸ **The auxiliary losses in OFA-MAS also contribute to performance improvement.** As we can see in the results, removing the balance or gate sparsity losses leads to a noticeable drop in accuracy, indicating their role in stabilizing training and enhancing generative learning.

4.5 Robustness Analysis

To assess robustness, we evaluate all methods under adversarial conditions by simulating compromised agent scenarios. Specifically, we replace a key agent role (e.g., “Knowledgeable Exper”) with a malicious prompt designed to generate misleading information that leads to incorrect reasoning. Figure 3a presents the performance comparison before and after adversarial attacks across all baseline methods. We can see that our **OFA-MAS framework demonstrates exceptional resilience**, experiencing only a 2.2%

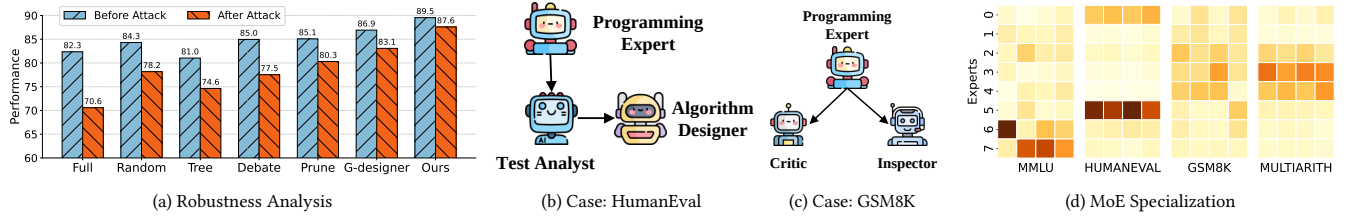


Figure 3: Comprehensive analysis including (a) robustness against adversarial attacks, (b-c) case study visualizations of generated topologies, and (d) MoE expert activation heatmaps across four domains showing clear specialization patterns.

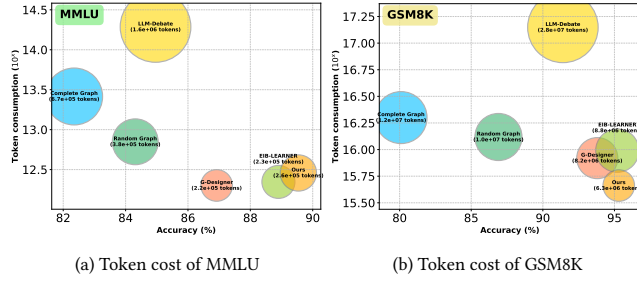


Figure 4: The prompt token cost comparison.

performance degradation (from 89.54% to 87.58%), significantly outperforming all other approaches. In contrast, fixed topology methods show substantial vulnerability, and learning-based methods like G-Designer (4.4% drop) and AgentPrune (5.6% drop) demonstrate moderate robustness but remain significantly more vulnerable than our approach. These results show that OFA-MAS maintains this advantage under adversarial scenarios, making it suitable for real-world deployment where agent reliability may be compromised.

4.6 Analysis of MoE Expert Specialization

To investigate how the MoE mechanism learns to specialize its experts for different task domains, we visualize the activation weights assigned by the gating network for a sample of tasks, presented in Figure 3d. The heatmaps provide compelling evidence by revealing two key phenomena: **① Intra-Domain Consistency.** Within a single domain, tasks consistently activate a similar set of experts. For example, across different “HumanEval” tasks, the activation patterns are remarkably stable, indicating that the model has learned a consistent and specialized strategy for code generation. **② Inter-Domain Specialization.** The preferred set of experts differs sharply across domains, demonstrating clear specialization. For instance, while **HumanEval** (code generation) heavily relies on a specialized subset (e.g., Experts 0 and 5), **MMLU** (general knowledge) activates a broader, different combination of experts (e.g., Experts 6 and 7). Similarly, mathematical reasoning domains like **GSM8K** and **MultiArith** converge on their own distinct signature, primarily utilizing Experts 2, 3, and 4.

4.7 Case Study

Figure 3b and Figure 3c visualize representative topologies, demonstrating OFA-MAS’s adaptive role selection. **① Domain-appropriate**

role selection. For the HumanEval coding task (Figure 3b), OFA-MAS selects code-generation-specific roles, forming a sequential refinement chain for iterative development. **② Flexible cross-domain role composition.** For the GSM8K math problem (Figure 3c), OFA-MAS selects math-relevant roles (Programming Expert, Inspector) and incorporates *Critic*—a role associated with MMLU knowledge tasks—for verification. This shows that OFA-MAS selects from a universal role pool rather than domain-specific roles, enabling more flexible collaboration design across task types.

4.8 Efficiency Analysis

Figure 4 illustrates the accuracy-efficiency trade-offs of different methods on MMLU and GSM8K, where bubble size represents token consumption. We observe that **OFA-MAS achieves superior accuracy with competitive efficiency.** On MMLU, OFA-MAS attains 89% accuracy using 2.6×10^5 tokens, comparable to learning-based baselines while outperforming them in accuracy. On GSM8K, it achieves 95% accuracy at the lowest token cost (6.3×10^6) among all competing methods. These results demonstrate that OFA-MAS not only generates effective collaborative topologies but also maintains computational efficiency.

5 Conclusion

In this paper, we introduced OFA-MAS, a one-for-all framework for generating adaptive collaboration topologies for multi-agent systems. By combining a task-aware graph state encoder (TAGSE) with a mixture-of-experts (MoE) architecture and a practical three-stage training strategy that progressively builds knowledge from general collaborative graphs to task-specific topologies, our model overcomes the limitations of the traditional one-for-one paradigm. Extensive experiments show that a universal model can not only generalize across diverse domains but can also leverage shared knowledge to achieve state-of-the-art performance. This work paves the way for more robust, scalable, and truly autonomous agentic systems, moving from specialized solutions towards a universal graph foundation model for MAS design.

Acknowledgments

The work of S. Pan and Q. Nguyen was partially supported by the Australian Research Council (ARC) under Grant No. DP240101547. The work of Y. Liu was partially supported by the ARC under Grant No. DE260101172.

References

- [1] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. 2024. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17682–17690.
- [2] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv:2107.03374* [cs.LG] <https://arxiv.org/abs/2107.03374>
- [3] Qingfeng Chen, Shiyuan Li, Yixin Liu, Shirui Pan, Geoffrey I Webb, and Shichao Zhang. 2025. Uncertainty-Aware Graph Neural Networks: A Multihop Evidence Fusion Approach. *IEEE Transactions on Neural Networks and Learning Systems* (2025).
- [4] Xiaohui Chen, Yinkai Wang, Jiaxing He, Yuanqi Du, Soha Hassoun, Xiaolin Xu, and Li-Ping Liu. 2025. Graph Generative Pre-trained Transformer. *arXiv preprint arXiv:2501.01073* (2025).
- [5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv:2110.14168* [cs.LG] <https://arxiv.org/abs/2110.14168>
- [6] Edo Cohen-Karlik, Eyal Rozenberg, and Daniel Freedman. 2024. Overcoming Order in Autoregressive Graph Generation. *arXiv preprint arXiv:2402.03387* (2024).
- [7] Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning*. 11733–11763.
- [8] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. *arXiv:2009.03300* [cs.CY] <https://arxiv.org/abs/2009.03300>
- [9] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiaowu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. 2024. MetaGPT: Meta programming for a multi-agent collaborative framework. *International Conference on Learning Representations*, ICLR.
- [10] Yue Hu, Yuzhuo Cai, Yaxin Du, Xinyu Zhu, Xiangrui Liu, Zijie Yu, Yuchen Hou, Shuo Tang, and Siheng Chen. 2025. Self-evolving multi-agent collaboration networks for software development. In *Thirteenth International Conference on Learning Representations*.
- [11] Akbar Khan, John Hughes, Dan Valentine, Laura Ruis, Kshitij Sachan, Ansh Radhakrishnan, Edward Grefenstette, Samuel R Bowman, Tim Rocktäschel, and Ethan Perez. 2024. Debating with more persuasive LLMs leads to more truthful answers. In *International Conference on Machine Learning*. 23662–23733.
- [12] Jindong Li, Yongguang Li, Yali Fu, Jiahong Liu, Yixin Liu, Menglin Yang, and Irwin King. 2026. CLIP-Powered Domain Generalization and Domain Adaptation: A Comprehensive Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2026).
- [13] Shiyuan Li, Yixin Liu, Qingfeng Chen, Geoffrey I Webb, and Shirui Pan. 2024. Noise-resilient unsupervised graph representation learning via multi-hop feature quality estimation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1255–1265.
- [14] Shiyuan Li, Yixin Liu, Qingsong Wen, Chengqi Zhang, and Shirui Pan. 2026. Assemble your crew: Automatic multi-agent communication topology design via autoregressive graph generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [15] Renjie Liao, Yujia Li, Yang Song, Shenlong Wang, Charlie Nash, William L Hamilton, David Duvenaud, Raquel Urtasun, and Richard Zemel. 2019. Efficient graph generation with graph recurrent attention networks. *Advances in Neural Information Processing Systems* 32 (2019).
- [16] Wang Ling, Dani Yogatama, Chris Dyer, and Phil Blunsom. 2017. Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems. *arXiv:1705.04146* [cs.AI] <https://arxiv.org/abs/1705.04146>
- [17] Yixin Liu, Shiyuan Li, Yu Zheng, Qingfeng Chen, Chengqi Zhang, and Shirui Pan. 2024. Arc: A generalist graph anomaly detector with in-context learning. *Advances in Neural Information Processing Systems* 37 (2024), 50772–50804.
- [18] Yixin Liu, Guibin Zhang, Kun Wang, Shiyuan Li, and Shirui Pan. 2026. Graph-augmented large language model agents: Current progress and future prospects. *IEEE Intelligent Systems* (2026).
- [19] Zijun Liu, Yanzhe Zhang, Peng Li, Yang Liu, and Diyi Yang. 2023. Dynamic llm-agent network: An llm-agent collaboration framework with agent team optimization. *arXiv preprint arXiv:2310.02170* (2023).
- [20] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. Gaia: a benchmark for general ai assistants. In *The Twelfth International Conference on Learning Representations*.
- [21] Rui Miao, Yixin Liu, Yili Wang, Xu Shen, Yue Tan, Yiwei Dai, Shirui Pan, and Xin Wang. 2025. Blindguard: Safeguarding llm-based multi-agent systems under unknown attacks. *arXiv preprint arXiv:2508.08127* (2025).
- [22] Zepeng Ning and Lihua Xie. 2024. A survey on multi-agent reinforcement learning and its application. *Journal of Automation and Intelligence* 3, 2 (2024), 73–91.
- [23] Junjun Pan, Yixin Liu, Rui Miao, Kaize Ding, Yu Zheng, Quoc Viet Hung Nguyen, Alan Wee-Chung Liew, and Shirui Pan. 2025. Explainable and Fine-Grained Safeguarding of LLM Multi-Agent Systems via Bi-Level Graph Anomaly Detection. *arXiv preprint arXiv:2512.18733* (2025).
- [24] Junjun Pan, Yixin Liu, Xin Zheng, Yizhen Zheng, Alan Wee-Chung Liew, Fuyi Li, and Shirui Pan. 2025. A label-free heterophily-guided approach for unsupervised graph fraud detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12443–12451.
- [25] Junjun Pan, Yixin Liu, Chuan Zhou, Fei Xiong, Alan Wee-Chung Liew, and Shirui Pan. 2026. Correcting False Alarms from Unseen: Adapting Graph Anomaly Detectors at Test Time. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [26] Junjun Pan, Yu Zheng, Yue Tan, and Yixin Liu. 2025. A Survey of Generalization of Graph Anomaly Detection: From Transfer Learning to Foundation Models. In *The 16th IEEE International Conference on Knowledge Graphs*.
- [27] Arkil Patel, Satwik Bhattamishra, and Navin Goyal. 2021. Are NLP Models really able to Solve Simple Math Word Problems?. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2080–2094.
- [28] Chen Qian, Zihao Xie, Yifei Wang, Wei Liu, Yufan Dang, Zhuoyun Du, Weize Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. 2025. Scaling large-language-model-based multi-agent collaboration. In *Thirteenth International Conference on Learning Representations*.
- [29] Subhro Roy and Dan Roth. 2016. Solving General Arithmetic Word Problems. *arXiv:1608.01413* [cs.CL] <https://arxiv.org/abs/1608.01413>
- [30] Xu Shen, Yixin Liu, Yiwei Dai, Yili Wang, Rui Miao, Yue Tan, Shirui Pan, and Xin Wang. 2025. Understanding the Information Propagation Effects of Communication Topologies in LLM-based Multi-Agent Systems. In *The Conference on Empirical Methods in Natural Language Processing*.
- [31] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.
- [32] Yue Tan, Xiaoqian Hu, Hao Xue, Celso De Melo, and Flora D Salim. 2025. Bisecl: Binding and Separation in Continual Learning for Video Language Understanding. In *Advances in Neural Information Processing Systems*.
- [33] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *arXiv:2203.11171* [cs.CL] <https://arxiv.org/abs/2203.11171>
- [34] Zhexuan Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. 2025. Agentdropout: Dynamic agent elimination for token-efficient and high-performance llm-based multi-agent collaboration. *arXiv preprint arXiv:2503.18891* (2025).
- [35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [36] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.
- [37] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafraan, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [38] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafraan, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- [39] Jiaxuan You, Zhitao Ying, Xiang Ren, William L Hamilton, and Jure Leskovec. 2018. GraphRNN: Generating Realistic Graphs with Deep Auto-regressive Models. In *International Conference on Machine Learning*. 5694–5703.
- [40] Guibin Zhang, Kaijie Chen, Guancheng Wan, Heng Chang, Hong Cheng, Kun Wang, Shuyue Hu, and Lei Bai. 2025. EvoFlow: Evolving Diverse Agentic Workflows On The Fly. *arXiv preprint arXiv:2502.07373* (2025).
- [41] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. 2025. Multi-agent Architecture Search via Agentic Supernet. In *International Conference on Machine Learning*.
- [42] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. 2025. Cut the crap: An economical communication pipeline for llm-based multi-agent systems. In *Thirteenth International Conference on Learning Representations*.
- [43] Guibin Zhang, Yanwei Yue, Xiangguo Sun, Guancheng Wan, Miao Yu, Junfeng Fang, Kun Wang, Tianlong Chen, and Dawei Cheng. 2025. G-designer: Architecting multi-agent communication topologies via graph neural networks. In *Forty-second International Conference on Machine Learning*.
- [44] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. 2022. Automatic chain of thought prompting in large language models. *arXiv preprint arXiv:2210.03493*

(2022).

- [45] Lingxiao Zhao, Xueying Ding, and Leman Akoglu. 2024. Pard: Permutation-Invariant Autoregressive Diffusion for Graph Generation. *Advances in Neural Information Processing Systems* 37 (2024).
- [46] Yunfeng Zhao, Yixin Liu, Shiyuan Li, Qingfeng Chen, Yu Zheng, and Shirui Pan. 2025. Freegad: A training-free yet effective approach for graph anomaly detection. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*. 4379–4389.
- [47] Changxi Zhu, Mehdi Dastani, and Shihan Wang. 2024. A survey of multi-agent deep reinforcement learning with communication. *Autonomous Agents and Multi-Agent Systems* 38, 1 (2024), 4.
- [48] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Jürgen Schmidhuber. 2024. Gptswarm: Language agents as optimizable graphs. In *International Conference on Machine Learning*.

Appendix

A Related Work

A.1 Multi-Agent Systems Topology Design

Multi-agent systems (MAS) have evolved from traditional reinforcement learning-based coordination [22, 32, 47] to sophisticated LLM-based collaboration frameworks, where topology design becomes crucial for effective task execution. Early approaches relied on manually crafted, domain-specific templates: sequential chains [35, 44] for step-by-step reasoning, tree-of-thought hierarchies [1, 37] for systematic exploration, and debate structures [7, 11, 19] for adversarial verification. While effective within their respective domains, these fixed patterns lack adaptability across diverse task requirements.

Recent advances have introduced learnable topology generation. Graph-based approaches such as AgentPrune [42] and AgentDrop [34] construct topologies by pruning dense communication graphs, while GPTSwarm [48] and G-Designer [43] employ graph neural networks for direct structure generation. Evolutionary methods [10, 40] and neural architecture search [41] have also shown promise in discovering optimal agent configurations. However, these approaches universally adopt the “one-for-one” paradigm, training specialized models for individual domains, which limits scalability and cross-domain knowledge transfer. Our work addresses this by proposing a unified “one-for-all” model with MoE-based specialization.

A.2 Autoregressive Graph Generation

Graph generation has evolved through multiple paradigms, with autoregressive methods emerging as a dominant approach due to their natural compatibility with sequential modeling. Early work like GraphRNN [39] established the foundation by generating graphs through sequential node and edge addition, while subsequent methods improved efficiency and expressiveness through better graph representations and neural architectures [15].

Recent advances have significantly expanded the capabilities of autoregressive graph generation. PARD [45] introduces permutation-invariant autoregressive diffusion that maintains effectiveness while addressing ordering sensitivity. Graph Generative Pre-trained Transformer (G2PT) [4] leverages transformer architectures for autoregressive graph generation with strong generalization capabilities. Methods like Orderless Regularization [6] tackle the fundamental ordering problem in sequential graph generation by encouraging invariance to different valid orderings.

B Dataset Details

B.1 In-Distribution Benchmarks

We evaluate OFA-MAS on six diverse benchmarks spanning multiple reasoning domains:

MMLU [8]. The Massive Multitask Language Understanding benchmark consists of 57 subjects across STEM, humanities, social sciences, and more. We use a subset of 153 test questions. Each question is multiple-choice with 4 options.

GSM8K [5]. A dataset of 8.5K high-quality linguistically diverse grade school math word problems. We use 1,000 problems from the test split. Each problem requires multi-step arithmetic reasoning.

AQuA [16]. Algebra Question Answering dataset contains 254 algebraic word problems with natural language rationales. Problems require algebraic reasoning with multiple solution steps. Each question is multiple-choice with 5 options.

MultiArith [29]. A collection of 600 elementary school-level arithmetic word problems requiring multi-step reasoning with numerical answers.

SVAMP [27]. Simple Variations on Arithmetic Math word Problems contains 1,000 problems with simple variations to test robustness. Problems are designed with subtle linguistic variations to challenge model generalization.

HumanEval [2]. A code generation benchmark with 164 programming problems. Each problem includes a function signature, docstring, body, and unit tests. Evaluated using Pass@1 metric.

B.2 Out-of-Distribution Benchmark

GAIA. The General AI Assistants benchmark is designed to evaluate AI assistants on real-world questions requiring multi-modal reasoning, tool use, and web search. We evaluate on three difficulty levels: Level-1 (53 questions), Level-2 (86 questions), and Level-3 (26 questions). These levels differ in complexity and reasoning requirements. Importantly, GAIA was never seen during training, making it a pure OOD generalization test.

Evaluation Protocol for GAIA. We adopt different evaluation protocols for one-for-all and one-for-one methods to ensure fair comparison:

- **One-for-All (OFA-MAS)**: We directly apply our unified model (trained on six in-distribution benchmarks) to GAIA without any additional training, demonstrating true zero-shot OOD generalization capability.
- **One-for-One Baselines**: Given GAIA’s knowledge-intensive nature and multi-modal reasoning requirements, we select the baseline models trained on MMLU (which shares similar knowledge-based characteristics) for GAIA evaluation. This provides baselines with the most relevant role configurations and domain knowledge for the OOD task.

This protocol highlights the key advantage of our one-for-all paradigm: a single unified model can generalize to unseen domains, while one-for-one methods require careful selection of which domain-specific model to apply.

B.3 Data Statistics

Table 4 provides comprehensive statistics of all datasets used in our experiments, including answer types, evaluation metrics, and licensing information.

Table 4: Dataset statistics and details.

Category	Dataset	Answer Type	Metric	#Train	#Test	License
General reasoning	MMLU	Multi-choice	Acc.	40	153	MIT License
Math reasoning	GSM8K	Number	Acc.	40	1,000	MIT License
	MultiArith	Number	Acc.	40	560	Unspecified
	SVAMP	Number	Acc.	40	960	MIT License
	AQuA	Multi-choice	Acc.	40	214	Apache-2.0
Code generation	HumanEval	Code	Pass@1	40	121	MIT License
OOD	GAIA (L1)	Multi-modal	Acc.	-	53	CC BY 4.0
	GAIA (L2)	Multi-modal	Acc.	-	86	CC BY 4.0
	GAIA (L3)	Multi-modal	Acc.	-	26	CC BY 4.0

C Baseline Details

Single-Agent Prompting Methods. We compare against three single-agent baselines: (1) **Vanilla**: Direct prompting without reasoning enhancement; (2) **Chain-of-Thought (CoT)** [35]: Prompts the LLM to generate intermediate reasoning steps with "Let's think step by step"; (3) **Self-Consistency (SC)** [33]: Samples 5 reasoning paths using CoT and selects the most consistent answer through majority voting.

Fixed Multi-Agent Topologies. We evaluate five hand-crafted topologies: (1) **Chain**: Sequential linear chain $A_1 \rightarrow A_2 \rightarrow \dots \rightarrow A_n$; (2) **Tree**: Hierarchical structure with multiple layers culminating in a root agent; (3) **Complete**: Fully connected graph enabling maximum information sharing; (4) **Random**: Randomly generated topology with random role assignments; (5) **LLM-Debate**: Debate-based topology where agents critique each other's reasoning over multiple rounds.

Learning-Based Topology Design (One-for-One). We compare against four SOTA learning-based methods, each trained separately per dataset: (1) **AgentPrune** [42]: Identifies and addresses communication redundancy by performing one-shot pruning on spatial-temporal message-passing graphs, yielding token-economic topologies while defending against adversarial attacks; (2) **Agent-Dropout** [34]: Inspired by dynamic role adjustment in management theory, identifies redundant agents and communications across different rounds by optimizing adjacency matrices, eliminating them to enhance token efficiency and task performance; (3) **G-Designer** [43]: Employs a variational graph auto-encoder to encode agents and task-specific virtual nodes, dynamically designing task-adaptive communication topologies customized for each task's difficulty and requirements; (4) **EIB-LEARNER** [30]: Uses a causal framework to analyze error propagation patterns and designs topologies by fusing connectivity patterns from both dense and sparse graphs, balancing error suppression with beneficial information diffusion.

D Implementation Details

D.1 Model Architecture

Task Encoder. We use Sentence-BERT (all-MiniLM-L6-v2) as the pre-trained sentence transformer, which outputs 384-dimensional embeddings. The task encoder MLP projects these to $d_{\text{task}} = 128$ dimensions through a 2-layer network with hidden dimension 256.

TAGSE. The Task-Aware Graph State Encoder uses $L = 4$ layers with hidden dimension $d_h = 256$. Each layer employs context-gated message passing with role-aware attention aggregation. Node features are initially projected from 384 dimensions to 256 dimensions.

MoE Module. We use $K = 8$ expert networks. Each expert consists of a 2-layer MLP with hidden dimension 256 and ReLU activation. The gating network is a 2-layer MLP: $128 \rightarrow 256 \rightarrow 8$ with ReLU and Softmax.

Role Pool. Our universal role pool $\mathcal{R}_{\text{OFA}}$ contains 19 diverse roles spanning multiple domains, including: Mathematician, Psychologist, Doctor, Algorithm Designer, Programming Expert, Inspector, etc. Additionally, we define START and END tokens for autoregressive generation. This is significantly larger than domain-specific role pools used in one-for-one methods (typically 4-6 roles per domain).

D.2 Training Configuration

Stage 1: Unconditional Pre-training.

- Dataset: $\mathcal{D}_{\text{graphs}}$ with 800 canonical graph topologies
- Topologies: Chain, Star, Tree, Layered, Complete (various sizes: 3-10 nodes)
- Epochs: 50
- Batch size: 32
- Learning rate: 2×10^{-3} (0.002)
- Optimizer: Adam
- Scheduler: ReduceLROnPlateau (factor=0.5, patience=10)

Stage 2: LLM-Guided Conditional Training.

- Dataset: $\mathcal{D}_{\text{gen}}$ with 1,000 synthetic (query, topology) pairs
- LLM for data synthesis: GPT-4o-mini with temperature 0.7
- Domains: Knowledge QA, Math reasoning, Code generation, Logical reasoning, etc.
- Initialization: Load pre-trained weights from Stage 1
- Epochs: 50
- Batch size: 32
- Learning rate: 2×10^{-3} (0.002)
- Optimizer: Adam
- Scheduler: ReduceLROnPlateau (factor=0.5, patience=10)

Stage 3: Supervised Fine-tuning.

- Dataset: $\mathcal{D}_{\text{fine-tune}}$ with 1,000 empirically validated (query, topology) pairs (40 queries per benchmark $\times$ 6 benchmarks)
- Initialization: Load pre-trained weights from Stage 2

- Classical topologies tested: Chain, Layered, Star, Mesh, FullConnected
- Selection criterion: Top-2 performing topologies per query based on validation accuracy
- Epochs: 50
- Batch size: 32
- Learning rate: 5×10^{-4} (0.0005)
- Optimizer: Adam
- Scheduler: ReduceLROnPlateau (factor=0.5, patience=10)

Core Generation Instructions (Abbreviated)

Query Diversity: Generate queries across domains including code implementation, mathematical reasoning, multiple-choice questions, knowledge-based Q&A, medical consultation, and psychology.

Balanced Configuration:

- Agent count: Evenly distribute configurations for 2-6 agents
- Topology variety: Use all five topology types (Chain, Star, Mesh, FullConnected, Layered)
- Comprehensive role coverage: Use all 19 roles from the universal role pool

DAG Constraint: All generated topologies must be Directed Acyclic Graphs (no cycles allowed).

Redundancy Avoidance:

- For 2 agents: Only use Chain topology
- For 3 agents: Do NOT use FullConnected (identical to Mesh)
- For ≥ 4 agents: All five topologies are distinct

Output Format: JSONL (one JSON object per line) with fields: query, agent_count, roles, topology, edges.

D.3 Loss Function Hyperparameters

The total training loss is a weighted combination of three components:

- MoE balance loss weight: $\lambda_{\text{balance}} = 0.2$
- Gate sparsity (L1) loss weight: $\lambda_{\text{gate}} = 0.1$

Total loss: $\mathcal{L} = \mathcal{L}_{\text{graph}} + \lambda_{\text{balance}} \mathcal{L}_{\text{balance}} + \lambda_{\text{gate}} \mathcal{L}_{\text{gate}}$

D.4 Inference Configuration

- Maximum graph size: 6 nodes (acts as safeguard to prevent over-generation)
- Minimum graph size: 2 nodes (END token sampling suppressed until this threshold)
- Generation strategy: Greedy decoding (selecting highest probability at each step) or multinomial sampling
- Early stopping: Generation terminates when model predicts END token
- Edge generation: Probabilistic sampling with Bernoulli distribution; if no edges sampled, force connection to highest-probability node
- MAS execution: $K = 1$ rounds of inter-agent communication
- Base LLM for agents: GPT-4o with temperature 0.0

D.5 Computational Resources

The experiments are conducted on a machine with 12 vCPU Intel(R) Xeon(R) Platinum 8255C CPU @ 2.50GHz and a single RTX 2080 Ti with 11GB GPU memory. The operating system of the machine is Ubuntu 20.04. As for software versions, we use Python 3.8, Pytorch 2.1.2, and CUDA 12.1.

E LLM Prompt Templates

E.1 Prompt for LLM-Driven Data Synthesis (Stage 2)

We use GPT-4o with temperature 0.7 to generate diverse (query, topology) pairs. The system prompt instructs the LLM to generate queries spanning multiple domains (code implementation, mathematical reasoning, multiple-choice questions, knowledge-based Q&A, medical consultation, psychology) and propose suitable MAS configurations using roles from our universal role pool and topologies from {Chain, Star, Mesh, FullConnected, Layered}. Key instructions include:

Example Output:

```
{ "query": "def odd_count(lst)...", "agent_count": 4,
  "roles": ["Project Manager", "Algorithm Designer",
            "Programming Expert", "Test Analyst"],
  "topology": "Chain", "edges": "0->1 1->2 2->3" }
```

E.2 Agent Role Prompts

Our universal role pool contains 19 specialized roles. Representative role prompts include:

Knowlegable Expert

You are a knowlegable expert in question answering. Please give several key entities that need to be searched in wikipedia to solve the problem. If there is no entity in the question that needs to be searched in Wikipedia, you don't have to provide it.

Mathematician

You are a mathematician who is good at math games, arithmetic calculation, and long-term planning.

Algorithm Designer

You are an algorithm designer. You need to specify the specific design of the algorithm, including the classes that may be defined and the functions used. When the implementation logic is complex, you can give the pseudocode logic of the main algorithm. I hope your reply will be more concise. Preferably within fifty words.

The complete role pool includes: Knowledgeable Expert, Critic, Mathematician, Psychologist, Historian, Doctor, Project Manager, Algorithm Designer, Programming Expert, Test Analyst, Bug Fixer, Math Solver, Mathematical Analyst, Programming Expert for Math, Inspector, and domain-specific variants for choice questions.